

Exercise 1: Configuring a Basic Spring Application

Scenario:

Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.

Steps:

1. Set Up a Spring Project:

- Create a Maven project named **LibraryManagement**.
- Add Spring Core dependencies in the **pom.xml** file.

2. Configure the Application Context:

- Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
- Define beans for **BookService** and **BookRepository** in the XML file.

3. Define Service and Repository Classes:

- Create a package **com.library.service** and add a class **BookService**.
- Create a package **com.library.repository** and add a class **BookRepository**.

4. Run the Application:

- Create a main class to load the Spring context and test the configuration.

1. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.library</groupId>
```

```
<artifactId>LibraryManagement</artifactId>
```

```
<version>1.0-SNAPSHOT</version>
```

```
<properties>
  <maven.compiler.source>25</maven.compiler.source>
  <maven.compiler.target>25</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.3.30</version>
  </dependency>
</dependencies>

</project>
```

2. applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="bookRepository"
    class="com.library.repository.BookRepository"/>

  <bean id="bookService"
    class="com.library.service.BookService"/>

</beans>
```

3. BookRepository.java

```
package com.library.repository;

public class BookRepository {
```

```
public BookRepository() {  
    System.out.println("BookRepository Bean Created");  
}  
  
public void displayRepository() {  
    System.out.println("Book Repository is Working");  
}  
}
```

4. BookService.java

```
package com.library.service;  
  
public class BookService {  
  
    public BookService() {  
        System.out.println("BookService Bean Created");  
    }  
  
    public void displayService() {  
        System.out.println("Book Service is Working");  
    }  
}
```

5. MainApp.java

```
package com.library;  
  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
import com.library.repository.BookRepository;  
import com.library.service.BookService;  
  
public class MainApp {  
  
    public static void main(String[] args) {  
  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext("applicationContext.xml");
```

```
    BookService service = context.getBean("bookService", BookService.class);
    BookRepository repository = context.getBean("bookRepository",
BookRepository.class);

    service.displayService();
    repository.displayRepository();
}
}
```

OUTPUT :



```
Run MainApp x
"C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=49675" "-Dfile.encoding=UTF-8"
BookRepository Bean Created
BookService Bean Created
Book Service is Working
Book Repository is Working
Process finished with exit code 0
```

Exercise 2: Implementing Dependency Injection

Scenario:

In the library management application, you need to manage the dependencies between the **BookService** and **BookRepository** classes using Spring's IoC and DI.

Steps:

1. Modify the XML Configuration:

- Update **applicationContext.xml** to wire **BookRepository** into **BookService**.

2. Update the **BookService** Class:

- Ensure that **BookService** class has a setter method for **BookRepository**.

3. Test the Configuration:

- Run the **LibraryManagementApplication** main class to verify the dependency injection.

1. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.library</groupId>
```

```
<artifactId>LibraryManagement</artifactId>
```

```
<version>1.0-SNAPSHOT</version>
```

```
<properties>
```

```
  <maven.compiler.source>25</maven.compiler.source>
```

```
  <maven.compiler.target>25</maven.compiler.target>
```

```
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
</properties>
```

```
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.3.30</version>
  </dependency>
</dependencies>

</project>
```

2. applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">

  <bean id="bookRepository"
    class="com.library.repository.BookRepository"/>

  <bean id="bookService"
    class="com.library.service.BookService">

    <property name="bookRepository" ref="bookRepository"/>

  </bean>

</beans>
```

3. BookRepository.java

```
package com.library.repository;

public class BookRepository {

  public BookRepository() {
    System.out.println("BookRepository Bean Created");
  }
}
```

```
}

public void displayRepository() {
    System.out.println("Book Repository is Working");
}
}
```

4. BookService.java

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public BookService() {
        System.out.println("BookService Bean Created");
    }

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void displayService() {
        System.out.println("Book Service is Working");
        bookRepository.displayRepository();
    }
}
```

5. MainApp.java

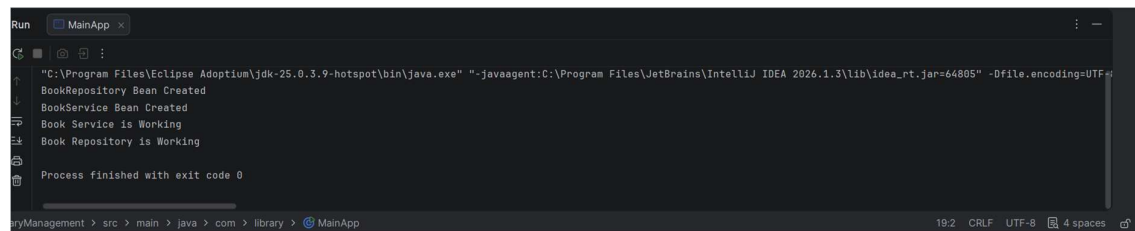
```
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;
```

```
public class MainApp {  
  
    public static void main(String[] args) {  
  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext("applicationContext.xml");  
  
        BookService service = context.getBean("bookService", BookService.class);  
  
        service.displayService();  
    }  
}
```

OUTPUT :



```
Run MainApp x  
"C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=64805" -Dfile.encoding=UTF-8  
BookRepository Bean Created  
BookService Bean Created  
Book Service is Working  
Book Repository is Working  
Process finished with exit code 0  
LibraryManagement > src > main > java > com > library > MainApp 19:2 CRLF UTF-8 4 spaces
```


Exercise 4: Creating and Configuring a Maven Project

Scenario:

You need to set up a new Maven project for the library management application and add Spring dependencies.

Steps:

1. Create a New Maven Project:

- Create a new Maven project named **LibraryManagement**.

2. Add Spring Dependencies in pom.xml:

- Include dependencies for **Spring Context**, **Spring AOP**, and **Spring WebMVC**.

3. Configure Maven Plugins:

- Configure the **Maven Compiler Plugin** for **Java version 1.8** in the **pom.xml** file.

To create and configure a Maven project by adding Spring Context, Spring AOP, and Spring WebMVC dependencies and configuring the Maven Compiler Plugin for Java 1.8.

Source Code

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.library</groupId>
```

```
<artifactId>LibraryManagement</artifactId>
```

```
<version>1.0-SNAPSHOT</version>
```

```
<properties>
```

```
<maven.compiler.source>1.8</maven.compiler.source>
```

```
<maven.compiler.target>1.8</maven.compiler.target>
```

```
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
```

```
</properties>
```

<dependencies>

<!-- Spring Context -->

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-context</artifactId>

<version>5.3.30</version>

</dependency>

<!-- Spring AOP -->

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-aop</artifactId>

<version>5.3.30</version>

</dependency>

<!-- Spring Web MVC -->

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-webmvc</artifactId>

<version>5.3.30</version>

</dependency>

</dependencies>

<build>

<plugins>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

<version>3.11.0</version>

<configuration>

<source>1.8</source>

<target>1.8</target>

</configuration>

</plugin>

```
</plugins>
</build>
```

```
</project>
```

OUTPUT:

